

C-programming 2022 Solution

Group-A(1 marks questions)

Fill up

- a) C compiler follows procedural approach.
- b) A header file keeps declarations of library functions.
- c) '2xy' is valid variable name in C compiler. False (True/False)
- d) A declaration: float a, b; occupies 8 Bytes Bytes of memory.
- e) Operators in C have precedence and it determines which operator is evaluated first.
- f) An instruction: a*=3; means a= a * 3.
- g) Scanf() can't read spaces string, gets(), gets() can do it.
- h) IDE provides editor , compiler and debugger.
- i) What is the answer of : -5%-3? -2.
- j) A control character for an address is %p.

Group-B(5 marks questions)

2) Mention difference between pre-increment and post-increment with suitable

Example.

Ans)

1. Pre-increment (`++x`):

- In pre-increment, the value of the variable is incremented before its current value is used in the expression.
- It means that the variable is incremented first, and then its incremented value is used in the expression.
- Example:

```
c Copy code  
  
#include <stdio.h>  
  
int main() {  
    int x = 5;  
    int result = ++x * 2;  
    printf("Pre-increment: %d\n", result); // Output: 12 (6 * 2)  
    return 0;  
}
```

2. Post-increment (`x++`):

- In post-increment, the value of the variable is incremented after its current value is used in the expression.
- It means that the variable's current value is used in the expression first, and then it is incremented.
- Example:

```
c Copy code  
  
#include <stdio.h>  
  
int main() {  
    int x = 5;  
    int result = x++ * 2;  
    printf("Post-increment: %d\n", result); // Output: 10 (5 * 2)  
    return 0;  
}
```

3) Differentiate 'auto' and 'static' storage classes with examples.

Ans)

Aspect	`auto` Storage Class	`static` Storage Class
Scope	Local to the block where it is declared.	Local to the block where it is declared.
Lifetime	Created when the block is entered and destroyed when the block exits.	Created when the program starts and destroyed when the program terminates.
Initialization	Automatically initialized to garbage value if not explicitly initialized.	Automatically initialized to zero if not explicitly initialized.
Memory Allocation	Allocated on the stack.	Allocated in the data segment of memory.

Example	<pre>// Example of auto storage class void foo() { auto int x; // auto variable de }</pre>	<pre>// Example of static storage class void bar() { static int y; // static variable de }</pre>
---------	--	--

4) Explain the basic string standard functions with proper examples.

Ans) Given below are some of the basic string standard functions in C that are commonly used for string manipulation. Each function serves a specific purpose and can be applied to various scenarios involving strings. These are-

1. strlen():

- Function: Calculates the length of a string (number of characters excluding the null terminator).
- Prototype: `size_t strlen(const char *str);`
- Example:

```
c Copy code

#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello, world!";
    int length = strlen(str);
    printf("Length of the string: %d\n", length); // Output: 13
    return 0;
}
```

2. strcpy():

- Function: Copies the content of one string to another.
- Prototype: ``char *strcpy(char *dest, const char *src);``
- Example:

```
c Copy code  
  
#include <stdio.h>  
#include <string.h>  
  
int main() {  
    char source[] = "Hello";  
    char destination[20];  
    strcpy(destination, source);  
    printf("Copied string: %s\n", destination); // Output: Hello  
    return 0;  
}
```

3. strcat():

- Function: Concatenates (appends) one string to the end of another.
- Prototype: ``char *strcat(char *dest, const char *src);``
- Example:

```
c Copy code  
  
#include <stdio.h>  
#include <string.h>  
  
int main() {  
    char str1[20] = "Hello, ";  
    char str2[] = "world!";  
    strcat(str1, str2);  
    printf("Concatenated string: %s\n", str1); // Output: Hello, world!  
    return 0;  
}
```

4. strcmp():

- Function: Compares two strings lexicographically.
- Prototype: `int strcmp(const char *str1, const char *str2);`
- Example:

```
c Copy code

#include <stdio.h>
#include <string.h>

int main() {
    char str1[] = "apple";
    char str2[] = "banana";
    int result = strcmp(str1, str2);
    if (result < 0)
        printf("%s comes before %s\n", str1, str2); // Output: apple comes before
    else if (result > 0)
        printf("%s comes after %s\n", str1, str2);
    else
        printf("%s is equal to %s\n", str1, str2);
    return 0;
}
```

5) How while() and do-while() loops are different from one another? Explain with example.

Ans)

The `while` and `do-while` loops are both used for repeating a block of code while a specified condition is true. However, they differ in terms of when the condition is checked. Here's a detailed explanation along with examples:

Aspect	while Loop	do-while Loop
Condition Check	The condition is checked before the loop body is executed.	The condition is checked after the loop body is executed.
Execution Guarantee	The loop body may not execute at all if the condition is false initially.	The loop body is guaranteed to execute at least once, regardless of the condition.

Aspect	while Loop	do-while Loop
Syntax	<pre>while (condition) { // loop body // statements to be executed as long as condition is true }</pre>	<pre>do { // loop body // statements to be executed at least once } while (condition);</pre>
Example	<pre>#include <stdio.h> int main() { int count = 0; while (count < 5) { printf("Count is: %d\n", count); count++; } return 0; }</pre>	<pre>#include <stdio.h> int main() { int count = 0; do { printf("Count is: %d\n", count); count++; } while (count < 5); return 0; }</pre>

6) Write a C-program to insert some integer values in an array and then print the average of them.

Ans)

```

#include <stdio.h>

int main() {
    int n, sum = 0;
    float average;

    // Read the number of elements
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    // Declare an array to store the integers
    int arr[n];

    // Insert values into the array
    printf("Enter %d integers:\n", n);
    for(int i = 0; i < n; i++) {
        printf("Element %d: ", i + 1);
        scanf("%d", &arr[i]);
    }

    // Calculate the sum of the values
    for(int i = 0; i < n; i++) {
        sum += arr[i];
    }

    // Calculate the average
    average = (float)sum / n;

    // Print the average
    printf("The average of the entered values is: %.2f\n", average);

    return 0;
}

```

7) Write a c program to find out maximum and minimum number out of three integers values by using conditional operator.

Ans)

```
#include <stdio.h>

int main() {
    int a, b, c;
    int max, min;

    // Read three integer values
    printf("Enter three integers: ");
    scanf("%d %d %d", &a, &b, &c);

    // Finding maximum value using conditional operator
    max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);

    // Finding minimum value using conditional operator
    min = (a < b) ? ((a < c) ? a : c) : ((b < c) ? b : c);

    // Print the maximum and minimum values
    printf("Maximum value: %d\n", max);
    printf("Minimum value: %d\n", min);

    return 0;
}
```

8) Write down outputs for the program below:(Give explanation)

a) #include <stdio.h>

#include <conio.h>

int main()

{

int i,j;

for(i=1; i<4;i++)

for(j=1; j<3; j++){

if(i<=j)

break;

printf("%d%d\n",i,j);

} return 0;

}

b) #include <stdio.h>

#include <conio.h>

int main()

{

int i,j;

for(i=1; i<4;i++)

for(j=1; j<3; j++){

if(i==j)

continue;

printf("%d%d\n",i,j);

} return 0;

}

Ans) Part-a

Explanation and Output:

- The outer loop runs with `i` taking values 1, 2, and 3.
- The inner loop runs with `j` taking values 1 and 2.
- The condition `if (i <= j)` determines whether to break out of the inner loop.
- If `i` is less than or equal to `j`, the inner loop breaks and moves to the next iteration of the outer loop.
- If `i` is greater than `j`, the `printf` statement is executed.

Output:

```
Copy code

21
31
32
```

Part-b

Explanation and Output:

- The outer loop runs with `i` taking values 1, 2, and 3.
- The inner loop runs with `j` taking values 1 and 2.
- The condition `if (i == j)` determines whether to continue to the next iteration of the inner loop.
- If `i` is equal to `j`, the `continue` statement skips the `printf` statement and moves to the next iteration of the inner loop.
- If `i` is not equal to `j`, the `printf` statement is executed.

Output:

```
Copy code

12
21
31
32
```

Group-C

(10 marks questions)

9) a) Define Recursion. Write a program in C to find out Greatest Common Divisor between two integer values by using recursion.

b) Write a C program to print the following pattern :(5+5=10 marks)

```
1
1 2 3
1 2 3 4 5
```

Ans)

Part-a

Definition of Recursion:

Recursion is a programming technique where a function calls itself directly or indirectly to solve a problem. Each recursive call reduces the problem into smaller subproblems, eventually reaching a base case that terminates the recursion.

C Program to Find Greatest Common Divisor (GCD) Using Recursion:

```
c Copy code

#include <stdio.h>

// Function to find GCD using recursion
int gcd(int a, int b) {
    if (b == 0)
        return a;
    else
        return gcd(b, a % b);
}

int main() {
    int num1, num2;

    // Read two integer values
    printf("Enter two integers: ");
    scanf("%d %d", &num1, &num2);

    // Calculate GCD
    int result = gcd(num1, num2);

    // Print the result
    printf("Greatest Common Divisor of %d and %d is: %d\n", num1, num2, result);

    return 0;
}
```

Part-b

C Program to Print the Pattern:

```
c Copy code

#include <stdio.h>

int main() {
    int i, j, k;
    int rows = 3; // Number of rows in the pattern

    for (i = 1; i <= rows; i++) {
        // Print leading spaces
        for (k = rows; k > i; k--) {
            printf(" ");
        }
        // Print numbers
        for (j = 1; j <= 2 * i - 1; j++) {
            printf("%d ", j);
        }
        printf("\n");
    }

    return 0;
}
```

10) a) Write a C program to check whether a number is palindrome or not.

b) Define function. What are the advantages and disadvantages of using function? Give solution to overcome the disadvantages of using function. Explain with example.(4+1+2+3=10 marks)

Ans) Part-a

A palindrome is a number that remains the same when its digits are reversed. Here's a C program to check whether a number is a palindrome or not:

```
c Copy code

#include <stdio.h>

int main() {
    int num, reversedNum = 0, remainder, originalNum;

    // Read an integer from the user
    printf("Enter an integer: ");
    scanf("%d", &num);

    originalNum = num;

    // Reverse the digits of the number
    while (num != 0) {
        remainder = num % 10;
        reversedNum = reversedNum * 10 + remainder;
        num /= 10;
    }

    // Check if the original number is equal to the reversed number
    if (originalNum == reversedNum)
        printf("%d is a palindrome.\n", originalNum);
    else
        printf("%d is not a palindrome.\n", originalNum);

    return 0;
}
```

Part-b

Definition of Function:

A function is a block of code that performs a specific task or calculation. It encapsulates a set of instructions that can be executed repeatedly by calling the function with appropriate parameters. Functions help in modularizing code, improving code readability, reusability, and maintainability.

Advantages of Using Functions:

1. **Modularity:** Functions break down a complex problem into smaller, manageable tasks, making the code easier to understand and maintain.
2. **Reusability:** Once defined, functions can be called multiple times from different parts of the program, reducing code duplication.
3. **Abstraction:** Functions hide the implementation details and provide a clean interface to the user, making the code easier to comprehend.
4. **Encapsulation:** Functions encapsulate related operations, promoting code organization and reducing the likelihood of errors.

Disadvantages of Using Functions:

1. **Overhead:** Function calls introduce overhead in terms of time and memory, especially for small, frequently called functions.
2. **Complexity:** Overuse of functions can lead to a proliferation of small functions, increasing code complexity and maintenance overhead.
3. **Dependency:** Functions can create dependencies between different parts of the code, making it harder to track and modify.
4. **Debugging:** Debugging can become more complex when the program flow involves multiple function calls, especially if the functions are deeply nested.

Solutions to Overcome Disadvantages:

1. **Inline Functions:** Use inline functions for small, frequently called functions to reduce the overhead of function calls.
2. **Function Optimization:** Optimize critical functions to minimize overhead and improve performance, for example, by using appropriate data structures or algorithms.
3. **Proper Function Design:** Design functions to have clear and single responsibilities, avoiding overly complex or tightly coupled implementations.
4. **Code Review:** Conduct code reviews to ensure that functions are well-designed, appropriately named, and correctly implemented.
5. **Documentation:** Provide comprehensive documentation for functions, including their purpose, parameters, return values, and usage examples, to aid understanding and usage.

Example:

```
c Copy code  
  
#include <stdio.h>  
  
// Function to calculate the factorial of a number  
int factorial(int n) {  
    if (n == 0 || n == 1)  
        return 1;  
    else  
        return n * factorial(n - 1);  
}  
  
int main() {  
    int num;  
  
    // Read an integer from the user  
    printf("Enter a number: ");  
    scanf("%d", &num);  
  
    // Call the factorial function and print the result  
    printf("Factorial of %d is: %d\n", num, factorial(num));  
  
    return 0;  
}
```

11) a) Discuss output differences between “Call by value” and “Call by reference” for swapping two integers numbers.

b) Write a program in C to sort the array (mentioned below) in ascending order-

34	23	45	4	67	17	7
----	----	----	---	----	----	---

(5+5=10 marks)

Ans) Part-A

Call by Value:

In Call by Value, copies of the actual parameters are passed to the function. Any modifications made to the parameters inside the function do not affect the original variables in the calling function.

Output:

```
mathematica Copy code  
  
Before swapping: num1 = 10, num2 = 20  
After swapping: num1 = 10, num2 = 20
```

- In Call by Value, the original variables remain unchanged as copies of their values are passed to the function. Therefore, swapping does not affect the original variables.

Call by Reference:

In Call by Reference, the addresses of the actual parameters are passed to the function. This allows the function to directly modify the values stored at those addresses, affecting the original variables in the calling function.

Output:

```
mathematica Copy code  
  
Before swapping: num1 = 10, num2 = 20  
After swapping: num1 = 20, num2 = 10
```

- In Call by Reference, the original variables are modified directly as their addresses are passed to the function. Therefore, swapping affects the original variables.

Part-b

c

 Copy code

```
#include <stdio.h>
#include <stdlib.h> // For qsort function

// Comparison function for qsort
int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int main() {
    int arr[] = {34, 23, 45, 4, 67, 17, 7};
    int n = sizeof(arr) / sizeof(arr[0]);

    // Print the original array
    printf("Original Array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // Sort the array in ascending order using qsort
    qsort(arr, n, sizeof(int), compare);

    // Print the sorted array
    printf("Sorted Array (Ascending Order): ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Output:

mathematica

 Copy code

```
Original Array: 34 23 45 4 67 17 7
Sorted Array (Ascending Order): 4 7 17 23 34 45 67
```

12) a) Write a note on – Switch and Case. What is the role of ‘break’ in switch-case? Explain with example.

b) Write a program in C to add first ‘n’ prime numbers and print the result. (Use separate function for prime numbers checking) (5+5=10 marks)

Ans) Part-a

`switch` and `case` are control flow statements in C used for decision-making. They allow you to execute different blocks of code based on the value of a given expression. The `switch` statement evaluates an expression and compares it against multiple values (cases), executing the block of code associated with the matching case.

Role of `break` in Switch-Case:

The `break` statement in a `switch` block is used to terminate the execution of the `switch` statement. When a `break` statement is encountered, the control exits the `switch` block, and subsequent code outside the `switch` block is executed.

Example:

```
c Copy code

#include <stdio.h>

int main() {
    int choice;
    printf("Enter a number (1-3): ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("You entered 1.\n");
            break;
        case 2:
            printf("You entered 2.\n");
            break;
        case 3:
            printf("You entered 3.\n");
            break;
        default:
            printf("Invalid choice.\n");
    }

    printf("Switch statement ends here.\n");

    return 0;
}
```

Explanation:

- In this example, the user is prompted to enter a number (`choice`) between 1 and 3.
- The `switch` statement evaluates the value of `choice` and compares it against different cases (`1`, `2`, `3`).
- If `choice` matches a case, the corresponding block of code is executed.
- The `break` statement is used to exit the `switch` block after executing the corresponding case.
- If `choice` does not match any case, the `default` case is executed.
- After the `switch` block, the code continues execution normally.

Part-b

```
#include <stdio.h>
#include <stdbool.h>

// Function to check if a number is prime
bool isPrime(int num) {
    if (num <= 1) {
        return false;
    }
    for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0) {
            return false;
        }
    }
    return true;
}

// Function to add the first 'n' prime numbers
int sumOfPrimes(int n) {
    int count = 0;
    int sum = 0;
    int num = 2; // Start checking from the first prime number

    while (count < n) {
        if (isPrime(num)) {
            sum += num;
            count++;
        }
        num++;
    }
    return sum;
}
```

```

int main() {
    int n;

    // Read the value of 'n' from the user
    printf("Enter the number of prime numbers to add: ");
    scanf("%d", &n);

    // Calculate the sum of the first 'n' prime numbers
    int result = sumOfPrimes(n);

    // Print the result
    printf("The sum of the first %d prime numbers is: %d\n", n, result);

    return 0;
}

```

13) a) Define Pointer. Write a program in C to get reverse of a string, using pointers.

b) Write short notes on – relational operators and logical operators in C programming. (2+5+3=10 marks)

Ans) Part-a

```

#include <stdio.h>
#include <stdbool.h>

// Function to check if a number is prime
bool isPrime(int num) {
    if (num <= 1) {
        return false;
    }
    for (int i = 2; i <= num / 2; i++) {
        if (num % i == 0) {
            return false;
        }
    }
    return true;
}

// Function to add the first 'n' prime numbers
int sumOfPrimes(int n) {
    int count = 0;
    int sum = 0;
    int num = 2; // Start checking from the first prime number

    while (count < n) {
        if (isPrime(num)) {
            sum += num;
            count++;
        }
        num++;
    }
    return sum;
}

```

```

int main() {
    char str[100];

    // Read a string from the user
    printf("Enter a string: ");
    gets(str);

    // Reverse the string using pointers
    reverseString(str);

    // Print the reversed string
    printf("Reversed string: %s\n", str);

    return 0;
}

```

Example Output:

```

c

Enter a string: hello
Reversed string: olleh

```

Part-b

Relational Operators in C

Relational operators in C are used to compare two values or expressions. The result of a relational operation is either true (non-zero) or false (zero). These operators are essential for decision-making and looping constructs.

Operator	Meaning	Example
<code>==</code>	Equal to	<code>a == b</code>
<code>!=</code>	Not equal to	<code>a != b</code>
<code>></code>	Greater than	<code>a > b</code>
<code><</code>	Less than	<code>a < b</code>
<code>>=</code>	Greater than or equal to	<code>a >= b</code>
<code><=</code>	Less than or equal to	<code>a <= b</code>

Logical Operators in C

Logical operators in C are used to combine multiple relational expressions. They return true or false based on the logical relationship between the expressions.

Common Logical Operators:

Operator	Meaning	Example
<code>&&</code>	Logical AND	<code>(a > b) && (a > c)</code>
<code> </code>	Logical OR	<code>(a > b) (a < c)</code>
<code>!</code>	Logical NOT	<code>!(a == b)</code>